



El objetivo académico de la práctica final del curso es consolidar los conocimientos que se han adquirido en clase. En concreto, los alumnos deberán demostrar, a través del desarrollo de un proyecto completo, el uso de los siguientes conceptos: planteamiento algorítmico, tipos de datos, claridad del código, estructuras de control, tratamiento secuencial, subprogramas, estructuras de datos, programación orientada a objetos, ficheros de texto, diseño descendente e interfaz a través del teclado, además de trabajar las competencias genéricas de la asignatura: análisis y síntesis, organización y toma de decisiones, y comunicar conceptos propios de la informática de manera escrita.

Para poder evaluar estos conocimientos, se define este ejercicio práctico que se resolverá con las mismas herramientas que se han utilizado durante el curso: el lenguaje de programación *Java*, el entorno integrado de desarrollo *NetBeans* (proyecto *Java Ant*), la clase *LT* y otras herramientas y clases que se hayan utilizado durante el curso o se desarrollen expresamente para este proyecto. No se aceptarán soluciones que utilicen conceptos avanzados no trabajados en clase.

La práctica puede hacerse de manera individual o en un grupo de dos miembros del mismo grupo de prácticas.

A continuación, se detalla el [enunciado de la práctica](#), algunas [restricciones](#), qué debe [entregarse](#), los [criterios generales de evaluación](#) y un [ejemplo de ejecución](#). Es importante destacar que no solo el programa debe funcionar correctamente, sino que, además, debe cumplir otros criterios de calidad, tal como se especifica en las restricciones y en los criterios generales de evaluación.

La fecha límite para entregar la práctica es el día del examen de cada convocatoria. Los alumnos deberán subir a Aula Digital, en un fichero comprimido, un informe del trabajo realizado y el proyecto *NetBeans* con el código fuente del programa desarrollado.

Es necesario aprobar esta práctica para aprobar la asignatura. Esta práctica es recuperable en el período de evaluación extraordinaria.

Según la normativa sobre fraude académico de la Universitat de les Illes Balears (Acuerdo Normativo 15418¹), se considera **fraude académico** (art. 5.1):

e. Incluir contenidos de cualquier fuente (**obras ajenas, páginas web, inteligencia artificial generativa...**) en los trabajos y las prácticas individuales o de grupo, y presentarlos de tal manera que **se hagan pasar como propios** cuando no lo son y no citarlos debidamente.

g. Entregar o **copiar partes de un trabajo** entregado por otro estudiante, el mismo año o en años anteriores, y presentarlo como un trabajo nuevo.

h. **Encargar la realización de un trabajo** a terceros.

Según el artículo 5.4:

4. La sanción se aplicará a **todos los estudiantes** que hayan colaborado en la comisión del fraude académico.

¹ <https://seu.uib.cat/fou/acord/15418/>

Según la normativa de del régimen disciplinario de los estudiantes de la Universitat de les Illes Balears (Acuerdo Normativo 15774²), este tipo de comportamientos son considerados **faltas graves**.

Las prácticas entregadas se revisarán, de manera automática, con un **software antiplagio** especialmente diseñado para detectarlo en código, que comprobará todas las entregas de este curso y los pasados.

En concreto, y de manera resumida, no podéis copiar código (ya sea de un compañero o de una IA), aunque luego “lo modifiquéis lo suficiente para que no se note”. En caso de sospecha, los profesores se reservan el derecho de convocar los estudiantes implicados a **revisión** para aclararlo y tomar las medidas correspondientes.

Enunciado

Introducción

Cifras y letras es un concurso de televisión en el cual se juegan un número determinado de rondas (por ejemplo, 10), que consisten, alternativamente, en dos subjuegos:

- ◆ **Rondas de letras:** se generan 10 letras. Cada participante debe proponer la palabra válida más larga que pueda formarse usando *solo* esas letras (no puede utilizarse una letra más veces de las que se ha generado).
- ◆ **Rondas de cifras:** se generan 6 números y un objetivo aleatorio. Con operaciones aritméticas básicas (+, −, ×, ÷) el participante intenta llegar exactamente al objetivo o lo más cerca posible.

Ambos jugadores intentan obtener la mejor solución con sus restricciones y objetivos y obtienen puntos según su solución. El jugador que ha acumulado más puntos al final de las rondas gana la partida.

En esta práctica el jugador humano debe poder jugar contra el ordenador (CPU).

El juego debe poderse jugar en castellano o en catalán (uno de los dos). El profesorado proporcionará una lista de letras y un diccionario de palabras válidas en cada idioma.

Objetivos

El objetivo de la práctica es desarrollar un programa que permita jugar a *Cifras y letras* y poder consultar un historial de las partidas jugadas.

En concreto, se plantean los siguientes objetivos detallados:

1. **Jugar una partida [obligatorio]**
 - a) Jugar contra el ordenador. [obligatorio]
 - Un jugador escribe su nombre (el ordenador se llama “CPU”).
 - Se juegan N rondas ($N \geq 2$; N par), alternando rondas de *cifras* y rondas de *letras*. Ambos jugadores compiten en cada ronda, cada uno con un juego de restricciones y objetivos. Se deben mostrar las restricciones

² <https://seu.uib.cat/fou/acord/15774/>

(cifras y objetivo, o letras), obtener las soluciones de cada jugador y acumular la puntuación obtenida.

- El ordenador debe intentar obtener la mejor solución posible.
- Puede ser interesante hacer que el ordenador juegue con las mismas restricciones que el humano, para ver si puede superarlo.

- Se muestra el vencedor de la partida según los puntos obtenidos.
- Se guarda un registro con los datos de la partida (véase punto 2).

b) Jugar contra otro jugador humano. [no obligatorio]

- Los dos jugadores escriben sus nombres.
- Se juegan N rondas ($N \geq 2$; N par), alternando rondas de *cifras* y rondas de *letras*. Ambos jugadores compiten en cada ronda, cada uno con un juego de restricciones y objetivos. Se deben mostrar las restricciones (cifras y objetivo, o letras), obtener las soluciones de cada jugador y acumular la puntuación obtenida.
 - En este caso, no tiene sentido que ambos jugadores jueguen con las mismas restricciones y objetivos, ya que el segundo tendría ventaja.
- Se muestra el vencedor de la partida según los puntos obtenidos.
- Se guarda un registro con los datos de la partida (véase punto 2).

2. Registro [obligatorio]

a) Mostrar resultados de las partidas. [obligatorio]

- Se muestran los datos registrados de todas las partidas.
 - Fecha de la partida (p. ej.: “2025-09-22 11:00:00”).
 - Tipo de partida (vs CPU, vs humano), si se implementan diferentes tipos de partida.
 - Nombre de ambos jugadores.
 - Nivel de la CPU, si se implementan niveles.
 - Número de rondas jugadas.
 - Puntuación de cada jugador.
 - Nombre del jugador ganador.

b) Mostrar estadísticas de un jugador. [no obligatorio]

- El usuario elige un nombre de jugador y se muestra un resumen de los datos registrados de ese jugador.
 - Número total de partidas jugadas, ganadas y % de ganadas.
 - Promedio de puntuación obtenida por partida.

3. Opciones [no obligatorio]

a) Permitir configurar el número de rondas, la cantidad de cifras y letras, idioma, etc.

- Todo esto es no obligatorio.

4. Salir [obligatorio]

- a) El programa consiste en la ejecución de este menú, de manera que permita jugar más de una partida o consultar los registros sin salir del programa hasta que el usuario introduzca que desea salir.

De estos objetivos, es **obligatorio** realizar, **como mínimo**, el 1.a (Jugar > **Jugar contra el ordenador**), el 2.a (Registro > **Mostrar resultados de las partidas**) y 4 (**Salir**).

Además, de manera opcional, se pueden introducir cambios al problema que en ningún caso pueden simplificar el enunciado o los objetivos anteriores. Por ejemplo, configurar el número de rondas, el orden de las rondas, introducir un número mínimo de vocales en las letras aleatorias, modos de juego adicionales, introducir niveles de dificultad en la inteligencia de la CPU, limitar el tiempo de respuesta de los jugadores, estadísticas más detalladas (palabras y cifras usadas), etc.

En cualquier caso, debe especificarse claramente en la memoria qué objetivos se han implementado, tanto de los descritos como parte del enunciado como los adicionales que se hayan podido introducir por parte de los estudiantes. No hacerlo puede hacer que no se tengan en cuenta para la calificación de la práctica.

Especificación del juego

Rondas de letras

- ◆ **Generación de letras:** 10 letras en total. Cada letra se obtiene, de manera aleatoria, de una bolsa con distribución aproximada al idioma en el que se juegue la partida. Los profesores proporcionarán un fichero con esta distribución, que debe ser procesado por el programa. Las letras pueden estar repetidas en la bolsa, pero no pueden sacarse dos veces de la bolsa. Por ejemplo: A N I M A L C O L R.
- ◆ **Validez de palabras:** una palabra es válida solo si cumple estas restricciones:
 1. Solo utiliza las letras disponibles, sin exceder su multiplicidad. Según el ejemplo anterior, “animal” es válida porque se han generado dos A, pero “color” no porque solo se había generado una O.
 2. Pertenece al diccionario de palabras válidas del idioma en el que se juegue la partida (proporcionado por los profesores).
- ◆ **Entrada del jugador:** el jugador introduce su propuesta de palabra.
 - ◇ De manera opcional, el programa debería permitir al usuario repetir este proceso hasta que introduzca una palabra válida (pero también dar la posibilidad de pasar).
 - ◇ Opcionalmente, puede introducirse una limitación de tiempo.
- ◆ **Estrategia de la CPU:** búsqueda por prueba de palabras del diccionario que puedan formarse con las letras dadas para encontrar la mejor disponible.
 - ◇ Opcionalmente, se puede comprobar solo un % de las palabras o escoger la palabra con una probabilidad según el nivel de dificultad. Debería tenerse en cuenta que es posible no encontrar una solución.
- ◆ **Puntuación:** una palabra de longitud L puntúa L puntos. Si algún jugador no encuentra una palabra válida, puntúa 0 puntos.

Rondas de cifras

- ◆ **Generación de números:** 6 números en total. Cada número se obtiene, de manera aleatoria, de una bolsa estándar (1–10, repetidos dos veces, y grandes 25, 50, 75, 100, repetidos una vez). Los profesores proporcionarán un fichero con esta distribución, que debe ser procesada por el programa. Los números pueden estar repetidos en la bolsa, pero no pueden sacarse dos veces. Por ejemplo: 6 2 6 3 5 50.
- ◆ **Objetivo:** un número entero entre 100 y 999, generado aleatoriamente. Por ejemplo, 498.
- ◆ **Reglas:** se pueden usar las operaciones enteras $+$, $-$, $\times$, $\div$, con cada número a lo sumo una vez; las operaciones deben producir enteros positivos; no puede usarse la división si el resultado no sería entero. Los resultados de estas operaciones pueden utilizarse para operaciones futuras. Con seis cifras iniciales, como mucho pueden llevarse a cabo cinco operaciones. Siguiendo el ejemplo anterior:

#	Cifras disponibles	Op.	O. 1	O. 2	Res.
1	6 2 6 3 5 50	$\times$	2	5	10
2	6 2 6 3 5 50 10	$\times$	50	10	500
3	6 2 6 3 5 50 10 500	$\div$	6	3	2
4	6 2 6 3 5 50 10 500 2	$-$	500	2	498
5	6 2 6 3 5 50 10 500 2 498	5ª operación no necesaria.			
Fin					

- ◆ **Entrada del jugador:** el jugador indica las operaciones que quiere llevar a cabo con las cifras disponibles. Los operadores se quitan de las cifras disponibles. Los resultados se añaden a las cifras disponibles. Al terminar las cinco operaciones o al señalar que no quiere hacer más operaciones, se selecciona como respuesta una de las cifras (disponible o no).
 - ◇ De manera opcional, puede darse la opción de repetir el proceso si no se llega a un resultado satisfactorio o se ha introducido un error.
 - ◇ Opcionalmente, puede introducirse una limitación de tiempo.
- ◆ **Estrategia de la CPU:** búsqueda aleatoria, con límite de intentos, del número objetivo. Se seleccionan dígitos y operaciones de manera aleatoria hasta agotar los dígitos disponibles y se vuelve a empezar. Si se prueba el proceso un número determinado de veces (por ejemplo, 1000), se rinde y debería quedarse con la mejor solución encontrada hasta entonces.
 - ◇ Para mostrar los cálculos, puede guardarse la salida que se mostraría por pantalla en una *String*. Al guardar la mejor solución, se guarda también esta *String*. Al final, se muestra la mejor solución con su cálculo.
- ◆ **Puntuación:** según la diferencia d entre el resultado y el objetivo:
 - ◇ $d = 0$: 10 puntos.

- ◇ $1 \leq d \leq 5$: 7 puntos.
- ◇ $6 \leq d \leq 10$: 5 puntos.
- ◇ $d > 10$: 0 puntos.

Ficheros

Cuanto a los ficheros que hagan referencia a un idioma (castellano o catalán), solo es obligatorio utilizar uno de los dos, a preferencia de los estudiantes. De manera opcional, pueden utilizarse ambos idiomas e, incluso, incorporar otros idiomas.

letras_es.txt	letras_ca.txt	cifras.txt
aaaaaaaaaaaaabbcccc	aaaaaaaaaaaaabbcccc	1 1 2 2 3 3 4 4 5
cdddddeeeeeeeeeeeee	dddeeeeeeeeeeeeeefg	5 6 6 7 7 8 8 9 9
fgghhiiiiiiijklllll	ghiiiiiiiiijklllllm	10 10 25 50 75
mmnnnnnnnoooooooooop	mmnnnnnnnooooooppqrr	100
pqrrrrrrrsssssstttt	rrrrrrrsssssstttt	
uuuuuvwxyz	tuuuuvwxyz	

Ejemplos de los ficheros de texto proporcionados por el profesorado. Por limitaciones de espacio, las líneas aparecen cortadas en este documento. El carácter salto de línea (' \n ') se representa con .

Los estudiantes pueden crear ficheros alternativos (con más o menos palabras, acentos, diferentes conjuntos de cifras y letras, etc.), pero la práctica **debe funcionar con los proporcionados por los profesores** (un idioma entre castellano o catalán es suficiente).

Bolsa de letras

Los profesores pondrán a disposición de los estudiantes un fichero con este contenido, por cada idioma: las letras que pueden aparecer en las rondas de letras, en una sola línea. Algunas letras están repetidas, y por ello pueden aparecer más de una vez en la ronda (véanse *letras_es.txt* y *letras_ca.txt*).

Diccionario de palabras

A la hora de comprobar si una palabra es correcta, deberá comprobarse si existe. Para ello, los profesores pondrán a disposición de los estudiantes un fichero con la lista de palabras válidas en cada idioma. Las palabras están separadas por un salto de línea (' \n ').

Bolsa de cifras

Los profesores pondrán a disposición de los estudiantes un fichero con las cifras que pueden aparecer en una ronda de cifras. Los números aparecen separados por espacios. Algunas cifras están repetidas, y por ello pueden aparecer más de una vez en la ronda (véase *cifras.txt*).

Fichero de resultados de las partidas

Los resultados de las partidas deben guardarse en un fichero de texto para poder consultarse posteriormente. El estudiante es libre de elegir el formato que prefiera, y a continuación se sugiere uno que podría ser adecuado. En este documento, se indica el carácter salto de línea (' \n ') en el fichero con .

partidas.txt
2025-09-22 09:00:00#vs CPU#Maria del Mar#CPU#10#2#12#20
2025-09-22 10:00:00#vs humano#Maria del Mar#Joan Antoni#0#4#24#16
2025-09-22 11:00:00#vs CPU#CPU#Maria del Mar#5#2#10#15

Descripción de los datos:

- ◆ Fecha: puede guardarse en formato texto o como *timestamp* numérico (a la fecha “2025-09-22 09:00:00” le corresponde el *timestamp* 1758524400).
- ◆ Tipo de partida: si no hay tipos de partida, puede obviarse. Puede ser en formato texto o de otro tipo. Tal vez puede combinarse con el nivel de CPU (= 0: vs humano, > 0: vs CPU).
- ◆ Nombre del jugador 1: texto, posiblemente con espacios.
- ◆ Nombre del jugador 2: texto, posiblemente con espacios.
- ◆ Nivel de la CPU: si no se implementa, puede obviarse.
- ◆ Número de rondas: número par de rondas de cifras y letras.
- ◆ Puntuación del jugador 1: número.
- ◆ Puntuación del jugador 2: número.

En ningún caso se permitirá generar un fichero redactado que contenga el texto final que se debe mostrar para el objetivo obligatorio 2.a y luego limitarse a mostrar directamente los caracteres o las líneas. Es **obligatorio** procesar el fichero de texto como *resultados* que contengan *campos*.

Restricciones

Para optar a aprobar la práctica se deben seguir las restricciones detalladas en esta sección y, como mínimo, implementar correctamente un **menú** (aunque haya opciones que muestren un mensaje que diga que aún no está implementado) con las **funcionalidades obligatorias** definidas en los Objetivos (1.a (Jugar > **Jugar contra el ordenador**), 2.a (Registro > **Mostrar resultados de las partidas**)) y 4 (**Salir**).

En concreto, es **obligatorio** hacer uso de los ficheros facilitados por los profesores: 1. bolsa de letras, 2. diccionario de palabras y 3. bolsa de cifras. Además, deben guardarse los resultados de las partidas y procesar este fichero de manera conceptual como *resultados* que contengan *campos*.

Se puede suponer que no hay ningún carácter fuera del rango definido por el código ASCII (p. ej.: caracteres acentuados, ç, ñ, etc.).

Se requiere el uso de la **Orientación a Objetos** para representar los datos necesarios para jugar las partidas y para procesar los ficheros. En concreto, el fichero de resultados contiene conceptualmente *resultados* que contienen *campos* y no pueden ser tratados directamente como *char[]* o *String*.

El uso de los métodos del tipo **String para el tratamiento secuencial de caracteres** está prohibido y supondrá un suspenso de la práctica con un 0. Se permite utilizar el tipo *String* únicamente para operaciones de entrada y salida de datos. Por ejemplo, **sí que se permiten** los literales *String*, concatenar *Strings*, mostrarlas por pantalla, leerlas de un buffer, usar el método *valueOf* para obtener la representación textual de un tipo primitivo, obtener la

representación en `char[]` con `toCharArray` y crear una a partir del constructor de `char[]`. Una lista no exhaustiva de los métodos de la clase `String` que **no se permiten** son: `charAt`, `contains`, `endsWith`, `equals`, `equalsIgnoreCase`, `indexOf`, `isEmpty`, `lastIndexOf`, `length`, `matches`, `replace`, `split`, `startsWith`, `substring`, `toLowerCase`, `toUpperCase`, `trim`. Tampoco se pueden emplear **métodos de la clase `Arrays`** o similares; por ejemplo, no se pueden emplear métodos como `arrayCopy`. Tampoco se permite utilizar técnicas o estrategias que no se hayan visto en clase (por ejemplo, *recursividad*, *backtracking* o *expresiones regulares*). Finalmente, tampoco pueden utilizarse estructuras más complejas que las trabajadas en clase, como estructuras de datos dinámicas (`ArrayList`, `LinkedList` y similares). La interacción con el teclado se llevará a cabo mediante la **clase `LT`** introducida en clase.

Se recuerda a los estudiantes que en esta asignatura solo se ha tratado una parte introductoria de la programación y de *Java*, que es un lenguaje muy extenso. El uso de aspectos avanzados no vistos en clase sin justificación se considerará un posible caso de **plagio académico**.

Se requiere que el programa esté implementado en **Java** en un proyecto **NetBeans Ant**. Si no es así, la práctica se calificará con un 0.

Se debe redactar un **informe breve** que resuma el proceso de desarrollo de este proyecto, según lo especificado en [Entrega](#). Si este no se entrega o está claramente incompleto, la práctica se calificará con un 0.

Entrega

Los alumnos dispondrán de un espacio en Aula Digital para entregar el trabajo realizado. Este espacio se cerrará el día del examen de cada convocatoria. Los estudiantes pueden elaborar este trabajo de manera individual o en equipo de dos, del mismo grupo de prácticas.

Los alumnos deben entregar, en un archivo comprimido, lo siguiente:

1. El **proyecto NetBeans (Java Ant)** con el código fuente del programa y todos los ficheros de texto necesarios para su funcionamiento, incluyendo un fichero de resultados con algunas partidas.
2. El **informe** en un documento PDF (3-6 páginas aproximadamente) que resuma el desarrollo de este proyecto. Debe contener, como mínimo: objetivos (obligatorios y opcionales), diseño descendente, estructuras de datos, algoritmos importantes, un breve manual de usuario, conclusiones y lista de referencias (con la mención explícita de qué uso se ha hecho de IA generativas).

Evaluación

La práctica final **no se evaluará** (resultando en un suspenso) en cualquiera de los casos especificados en el apartado de Restricciones.

En otro caso, la evaluación seguirá estos criterios de manera orientativa, en función de los objetivos que se hayan implementado.

◆ Informe:

- ◇ Contiene los apartados solicitados y están bien explicados.

◆ Diseño orientado a objetos:

- ◇ Se han creado las clases necesarias para solucionar el problema.
- ◇ La construcción de las clases, atributos y métodos es adecuada.
- ◆ **Programación de la solución:**
 - ◇ El código está bien indentado y organizado.
 - ◇ Se utilizan identificadores con significado.
 - ◇ El algoritmo es claro y se utilizan comentarios cuando es necesario.
 - ◇ Se utilizan subprogramas cuando es apropiado.
 - ◇ Se favorece el uso de funciones y parámetros en lugar de variables globales.
 - ◇ Se evita el malgasto de memoria al cargar ficheros grandes enteros en memoria (como el diccionario o el fichero de resultados).
- ◆ **Interfaz de usuario:**
 - ◇ El usuario sabe qué se espera de él y qué opciones tiene.
 - ◇ El usuario está informado de lo que sucede.
 - ◇ La presentación es ordenada y consistente.

Una práctica que cumpla las restricciones indicadas en Restricciones, donde se hayan implementado las **funcionalidades obligatorias** definidas en los Objetivos (1.a (Jugar > Jugar solo) y 2.a (Registro > Mostrar resultados de las partidas)) y que cumpla de manera suficiente los criterios anteriores se considerará aprobada. Si no se cumplen las restricciones, si no se implementan las funcionalidades obligatorias o si no se cumplen de manera suficiente los criterios anteriores, la práctica se considerará suspendida.

Una práctica suspendida en la convocatoria ordinaria **se puede recuperar en la extraordinaria**. Una práctica suspendida en la convocatoria extraordinaria **no se puede recuperar**. Se recuerda que **es necesario aprobar la práctica** para poder aprobar la asignatura.

Ejemplo de ejecución

El texto introducido por el usuario se marca así. El símbolo ↵ representa haber introducido la tecla de retorno (*enter*).

Éste es solo un ejemplo que muestra algunas de las funcionalidades.

```
*****
MENÚ PRINCIPAL
*****
  1 Jugar
  2 Registro
  s Salir
*****

Opción (1|2|s): 1↵

*****
JUGAR
*****
  1 Jugar contra el ordenador
  2 Jugar contra otro jugador
```

s Volver al menú principal

Opción (1|2|3|s):

JUGAR CONTRA EL ORDENADOR

Introduce el nombre del jugador:
Nombre del jugador 2: CPU.

Introduce cuántas rondas quieres jugar (número par):

Ronda 1 de 2: letras.
Turno de Maria del Mar.
Letras disponibles: a n i m a l c o l r
Introduce tu palabra:
Validando palabra:
- puede crearse con las letras disponibles
- existe en el diccionario
Longitud 3: +3 puntos.

Puntuaciones:
- Maria del Mar: 3 puntos.
- CPU: 0 puntos.

Turno de CPU.
Letras disponibles: a n i m a l c o l r
Palabra seleccionada por CPU: animal.
Longitud 6: +6 puntos.

Puntuaciones:
- Maria del Mar: 3 puntos.
- CPU: 6 puntos.

Ronda 2 de 2: cifras.
Turno de Maria del Mar.
Cifras disponibles: 6 2 6 3 5 50
Objetivo: 498.
Operación 1 (+|-|*|/=):
Operando 1 (6 2 6 3 5 50):
Operando 2 (6 6 3 5 50):
2 * 5 = 10

Cifras disponibles: 6 6 3 50 10
Objetivo: 498.
Operación 2 (+|-|*|/=):
Operando 1 (6 6 3 50 10):
Operando 2 (6 6 3 10):

$$50 * 10 = 500$$

Cifras disponibles: 6 6 3 500

Objetivo: 498.

Operación 3 (+|-|*|/|=):

Operando 1 (6 6 3 500):

Operando 2 (6 3 500):

- se ha comprobado que puede llevarse a cabo la división entera 6 / 3.

$$6 / 3 = 2$$

Cifras disponibles: 6 500 2

Objetivo: 498.

Operación 4 (+|-|*|/|=):

Operando 1 (6 500 2):

Operando 2 (6 2):

- se ha comprobado que puede llevarse a cabo la resta 500 - 2.

$$500 - 2 = 498$$

Cifras disponibles: 6 498

Objetivo: 498.

Operación 5 (+|-|*|/|=):

Introduce el resultado final (6 2 6 3 5 50 10 500 2 498):

Resultado exacto: +10 puntos.

Puntuaciones:

- Maria del Mar: 13 puntos.
- CPU: 6 puntos.

Turno de CPU.

Cifras disponibles: 6 2 6 3 5 50

Objetivo: 498.

Calculando...

Mejor resultado encontrado: 498.

Cifras disponibles: 6 2 6 3 5 50

Objetivo: 498.

Operación 1: $2 * 5 = 10$

Cifras disponibles: 6 6 3 50 10

Objetivo: 498.

Operación 2: $50 * 10 = 500$

Cifras disponibles: 6 6 3 500

Objetivo: 498.

Operación 3: $6 / 3 = 2$

Cifras disponibles: 6 500 2

Objetivo: 498.

Operación 4: $500 - 2 = 498$

Cifras disponibles: 6 498

Objetivo: 498.

Operación 5: hemos encontrado ya el resultado, nos paramos.

Cifras obtenidas: 6 2 6 3 5 50 10 500 2 498.

Buscamos el mejor resultado.

Resultado final: 498.

Resultado exacto: +10 puntos.

Puntuaciones:

- Maria del Mar: 13 puntos.

- CPU: 16 puntos.

Jugador ganador: CPU.

MENÚ PRINCIPAL

1 Jugar

2 Registro

s Salir

Opción (1|2|s):

REGISTRO

1 Mostrar resultados de las partidas

2 Mostrar estadísticas de un jugador

s Volver al menú principal

Opción (1|2|s):

DETALLES DE LAS PARTIDAS

Partida 1 (2025-09-22 09:00:00). Modo "vs CPU", 2 rondas,
ganador: "CPU".

- Jugador 1 "Maria del Mar": 12 puntos.

- Jugador 2 "CPU" (CPU nivel 10): 20 puntos.

Partida 2 (2025-09-22 10:00:00). Modo "vs humano", 4 rondas,
ganador: "Maria del Mar".

- Jugador 1 "Maria del Mar": 24 puntos.

- Jugador 2 "Joan Antoni": 16 puntos.

Partida 3 (2025-09-22 11:00:00). Modo "vs CPU", 2 rondas,
ganador: "Maria del Mar".

- Jugador 1 "CPU" (CPU nivel 5): 10 puntos.
- Jugador 2 "Maria del Mar": 15 puntos.

MENÚ PRINCIPAL

- 1 Jugar
- 2 Registro
- s Salir

Opción (1|2|s):

Hasta pronto!

[final de la ejecución del programa]